

למידה מאוחדת (Federated Learning) ברשתות רדיו קוגניטיביות --- CARLTON

תוכן העניינים

3	1 השיטה הקלאסית/הריכוזית (Centralized ML)
3	1.1 למה זה רע לרשת הרדיו שלך? (CARLTON)
3	2 הגישה של Learning: Federated המהפכה המתמטית
3	2.1 מה הנוסחה הזו אומרת בעצם?
3	3 למה זה קריטי למחקר שלך (Wireless) & (CARLTON)
4	4 המילון: מה אומרים הסימנים בפרויקט?
4	5 אנטומיה של סבב תקשורת (Federated Round)
5	6 הדילמה: תקשורת מול חישוב (Client Drift)
5	6.1 הסכנה: Drift Client (סטיית קליינט)
5	7 האתגר הגדול: Data Non-IID (מידע לא אחיד)
5	8 האתגר המרכזי: נתונים IID מול Non-IID
6	9 הפתרון: אלגוריתם (Federated FedAvg Averaging)
6	9.1 למה ממוצע "משוקלל" (Weighted Averaging)?
6	10 פירוק האלגוריתם צעד אחר צעד
6	10.1 מבט על: מי עושה מה?
7	10.2 שלב 0: האתחול (שרת)
7	10.3 שלב 1: הלופ הגלובלי (שרת)
7	10.4 שלב 2: הלופ המקומי במקביל (הסוכנים)
8	10.4.1 א. הורדת המודל הגלובלי
8	10.4.2 ב. לופ האפיזודות / האימון המקומי
9	10.4.3 ג. העלאה לשרת
9	10.5 שלב 3: האגרציה / שילוב התובנות (שרת)
9	10.6 האלגוריתם המלא במבט אחד
10	10.7 ציר זמן של סבב יחיד
10	10.8 מיפוי ישיר לקוד CARLTON
10	10.9 סיכום צעד-אחר-צעד: שרת מול סוכנים
11	11 הדילמה הגדולה: תקשורת לעומת סטיית קליינט

12	12 FedProx --- אופטימיזציה מאוחדת עם עונש קרבה
12	12.1 מוטיבציה: למה FedAvg נכשל ב-Non-IID ואיך FedProx מציל את המצב?
12	12.2 פונקציית המטרה המקומית
12	12.2.1 מה עושה הפרמטר μ (מיו)?
13	12.3 אינטואיציה: מודל "הגומייה"
13	12.4 מתי נרצה להשתמש ב-FedProx ברשת הרדיו?
13	12.5 פירוק האלגוריתם צעד אחר צעד (FedProx)
14	12.6 השתתפות חלקית --- פירוט נוסף
15	13 החיבור לקוד שלך

1 השיטה הקלאסית/הריכוזית (Centralized ML)

במצב הרגיל, אנחנו מניחים שיש לנו בסיס נתונים אחד גדול (D) (שיושב על שרת יחיד Lake (Data בענן). המטרה שלנו היא למצוא את המשקולות w של הרשת שיביאו למינימום את פונקציית ההפסד (השגיאה) הממוצעת על כל המידע:

$$\min_w F(w) = \frac{1}{|D|} \sum_{(x,y) \in D} \ell(w; x, y) \quad (1)$$

1.1 למה זה רע לרשת הרדיו שלך? (CARLTON)

רוחב פס מטורף (Bandwidth)

כדי לאמן מודל כזה, כל מנהל רשת בשטח צריך להזרים בכל מילי-שנייה את נתוני ה-SINR הגולמיים, תדרי השידור, ומטריצות הרעש לענן. ברשתות אלחוטיות זה אבסורד --- אנחנו מבזבזים את רוחב הפס היקר של הרשת רק כדי לשלוח נתונים על רוחב הפס!

אבטחה ופרטיות (Privacy)

נתוני הרדיו חושפים מיקומים מדויקים של משתמשים, דפוסי תנועה צבאיים או אסטרטגיות מסחריות של מפעילים (כמו סלקום ופרטנר). אף מפעיל לא יסכים להוציא את המידע הגולמי הזה מחוץ לשרת המקומי שלו.

2 הגישה של Learning: Federated המהפכה המתמטית

במקום לאסוף את המידע, אנחנו מבזרים את האופטימיזציה. המידע הגולמי (D_k) נשאר "נעול" בתוך כל סוכן/מנהל רשת k .

הנוסחה הגלובלית החדשה שרוצים למזער נראית כך:

$$\min_w F(w) = \sum_{k=1}^K \frac{n_k}{n} F_k(w) \quad (2)$$

כאשר פונקציית ההפסד המקומית של כל סוכן בנפרד היא:

$$F_k(w) = \frac{1}{n_k} \sum_{(x,y) \in D_k} \ell(w; x, y) \quad (3)$$

2.1 מה הנוסחה הזו אומרת בעצם?

המתמטיקה מוכיחה שאנחנו מנסים להגיע לאותה תוצאה בדיוק כאילו כל המידע היה במקום אחד, אבל בדרך חכמה יותר:

- השרת המרכזי (האגרגטור) לא רואה דגימות מידע (x, y) .
- כל סוכן k מחשב את ה-Loss המקומי שלו (F_k) על המכשיר שלו, מעדכן את המשקולות שלו, ושולח לשרת רק את השינוי במשקולות.
- השרת מחבר את כל המשקולות בצורה משוקללת (לפי גודל המידע $\frac{n_k}{n}$) כדי ליצור את המודל הגלובלי המאוחד.

3 למה זה קריטי למחקר שלך (Wireless) & CARLTON

הטקסט מסכם בצורה מדויקת שלוש סיבות שבגללן השילוב של CARLTON עם FL הוא פריצת דרך למחקר שלך:

1. **חיסכון עצום בתקשורת (Bandwidth):** משקולות של רשת עצבית קטנה (כמו ה-DeepMellow שלך) שוקלות כמה עשרות או מאות קילובייטים (KB). לשלוח קובץ כזה פעם ב-100 צעדים לשרת זה "חינמני" לחלוטין מבחינת עומס על הרשת, בהשוואה לשליחה של גיגה-בייטים (GB) של מדידות רדיו וספקטרום רציפות.

2. **עמידה ברגולציה ופרטיות:** המידע המקומי (ה-Buffer Replay של ה-SINR) לעולם לא עוזב את השרת של מנהל השרת המקומית. הגנה מושלמת על פרטיות המשתמשים וסודיות השרת.

3. **ביזור עומסי חישוב (Scalability):** במקום שרת ענן אחד יקר שיצטרך לעבד מיליוני אפיזודות של RL מכל הארץ, החישוב הפיזי (הסימולציה והרצת השרת על ה-SINR) מתחלק בין כל מנהלי השרתות בשטח. השרת המרכזי צריך רק לעשות פעולת חיבור וממוצע פשוטה, שהיא מהירה וזולה מאוד.

4 המילון: מה אומרים הסימנים בפרויקט?

K (מספר הקליינטים)

כמות רשתות התקשורת החופפות בסימולציה שלך (למשל, $K = 3$ עבור סלקום, פרטנר והרשת הצבאית).

w (המשקולות הגלובליות)

ה"מוח" המשותף והמעודכן ביותר של ה-DeepMellow שיושב בשרת המרכזי.

w_k (המשקולות המקומיות)

ה"מוח" של רשת k , אחרי שהיא התאמנה קצת לבד על נתוני ה-SINR שלה.

n_k (כמות המידע) כמות דגימות ה-SINR (או כמות האפיזודות) שרשת k אספה בבאפר המקומי שלה.

E (Local Epochs)

כמה צעדי אימון מנהל השרת עושה לבד בבית על הבאפר שלו, לפני שהוא רץ לספר לחבריה (לשרת).

T (סך כל הסיבובים)

כמה פעמים בסך הכל השרת והרשתות הולכים להחליף ביניהם משקולות לאורך כל הריצה.

5 אנטומיה של סבב תקשורת (Federated Round)

בכל סבב תקשורת t (מתוך T סבבים), מתרחש הריקוד הקבוע הבא:

1. **בחירת משתתפים (Client Selection):** השרת בוחר אילו רשתות משתתפות בסבב הנוכחי (בסימולציה שלך, לרוב נבחר את כולן, כלומר קבוצה S).

2. **שידור (Broadcast):** השרת שולח את המשקולות הנוכחיות שלו (w_t) לכל מנהלי השרתות. כולם מתחילים את הסבב עם אותו מוח בדיוק.

3. **חישוב מקומי (Local Computation):** כאן הקוד של Coordinator.py נכנס לפעולה. כל מנהל רשת מריץ את הלופ במשך E פעמים (Epochs) אוסף נתונים, ומעדכן את המודל המקומי שלו באמצעות `DeepMellow.learn()`. התוצאה היא משקולות חדשות: w_k^{t+1} .

4. **העלאה (Upload):** מנהלי השרתות שולחים לשרת (`TrainBlock.py`) רק את המשקולות החדשות שלהם.

5. **שילוב (Aggregation):** השרת משתמש באלגוריתם שנקרא FedAvg (קיצור של Federated Averaging). הוא עושה ממוצע משוקלל של המשקולות לפי כמות המידע n_k של כל רשת:

$$w_{t+1} = \sum_{k \in S} \frac{n_k}{n} w_k^{t+1} \quad (4)$$

6 הדילמה: תקשורת מול חישוב (Client Drift)

כדי שהמודל יתכנס, הוא צריך הרבה מידע. יש לנו שתי דרכים להשיג את זה, וכל אחת מגיעה עם מחיר:

אפשרות א' (תקשורת גבוהה)

לעשות אימון מקומי קצר מאוד ($E = 1$) ולשלוח מיד לשרת. המשמעות: הרשתות כל הזמן מסונכרנות, אבל אנחנו מציפים את ערוץ התקשורת בהעברות קבצים (משקולות של רשתות עצביות יכולות לשקול המון).

אפשרות ב' (חישוב גבוה)

להגיד למנהלי הרשתות להתאמן המון זמן לבד (E גדול) ורק אז לשלוח לשרת. זה חוסך המון רוחב פס בתקשורת.

6.1 הסכנה: Drift Client (סטיית קליינט)

כאן יש מלכודת מתמטית. אם נגדיר E גדול מדי, כל מנהל רשת יתאמן יותר מדי זמן על בעיות ה-SINR הספציפיות שלו.

ברשת קוגניטיבית זה מסוכן: רשת א' תפתח "תובנות קיצוניות" שטובות רק לה, וכאשר השרת ינסה לעשות ממוצע בין התובנות הקיצוניות של רשת א' לתובנות הקיצוניות של רשת ב', הממוצע יצא מודל גרוע שלא מתאים לאף אחת מהן! זה נקרא **Drift Client**.

7 האתגר הגדול: Data Non-IID (מידע לא אחיד)

בסטיסטיקה, **IID** אומר שהמידע של כולם מתפלג באותו אופן. אם הסביבה הייתה **IID**, כל רשת הייתה חווה בדיוק את אותן הפרעות ואותם משתמשים ראשיים (PUs). במצב כזה, הממוצע של FedAvg עובד כמו קסם ומתכנס למינימום המושלם. אבל במציאות של רדיו קוגניטיבי, המידע הוא **Non-IID**. רשת אחת יושבת ליד שדה תעופה וחווה הפרעות ממכ"ם בערוץ 1, ורשת שנייה יושבת במרכז העיר וחווה עומס של טלפונים בערוץ 4. הנוסחאות המקומיות שלהן (פונקציות ההפסד ℓ) מושכות לכיוונים שונים לחלוטין. מכיוון שהמידע הוא **Non-IID**, לעיתים ממוצע פשוט (FedAvg) לא יספיק, וזה בדיוק המקום שבו המחקר שלך יכול להתרחב בעתיד לאלגוריתמים מתקדמים יותר כמו **FedProx** (שמוסיף עונש מתמטי לסוכנים כדי שלא יתרחקו יותר מדי מהמודל הגלובלי). עכשיו, כשהבנו את הדינמיקה הזו של סבבי התקשורת ואת הסכנה של Drift Client בסביבה עם הפרעות רדיו שונות (**Non-IID**), אנחנו מבינים בדיוק למה אנחנו צריכים את הפרמטרים האלו בקוד.

8 האתגר המרכזי: נתונים IID מול Non-IID

בעולם מושלם, הלמידה המאוחדת הייתה קלה מאוד. בעולם האמיתי (וברדיו בפרט), הנתונים שכל מנהל רשת רואה הם שונים לחלוטין.

IID (מידע אחיד ומושלם) --- כל מנהלי הרשתות רואים בדיוק את אותה התפלגות של הפרעות ומשתמשים. אם הנתונים היו **IID**, ממוצע פשוט של המשקולות היה עובד כמו קסם בכל פעם.

Non-IID (מידע הטרוגני/מגוון) --- כל מנהל רשת חי ב"מציאות" משלו. מנהל רשת א' חווה רק הפרעות (מקביל למודל שרואה רק תמונות של כלבים), בעוד מנהל רשת ב' רואה ערוצים פנויים לחלוטין (מודל שרואה רק תמונות של חתולים).

כדי להבין איך זה פוגש אותך בקוד, הנה פירוט סוגי ה-**Non-IID** המרכזיים שקיימים בסביבת הרדיו שלך (CARLTON):

מה זה אומר בפרויקט הרדיו (CARLTON)?	המשמעות הכללית	סוג ה-Non-IID
רשת אחת יושבת באזור שקט (ערוצים עם SINR גבוה), ורשת אחרת באזור פקוק.	הטיה בתיוגים	skew Label
ערוץ 3 נראה שונה פיזיקלית (מבחינת תבנית הדעיכה) באזורים גיאוגרפיים שונים.	הטיה במאפיינים	skew Feature
לרשת א' יש 15 משתמשים (המון דגימות), ולרשת ב' יש רק 2 משתמשים (מעט דגימות).	הבדל בכמות המידע	skew Quantity
בזמן האימון, רשתות אחרות מחליפות תדרים, ולכן מפת ההפרעות משתנה ברגע.	הסביבה עצמה משתנה	drift Concept

9 הפתרון: אלגוריתם (Federated FedAvg Averaging)

כדי להתמודד עם הבעיות האלו, משתמשים באלגוריתם הקלאסי ביותר של גוגל: FedAvg. במקום שכל סוכן ישלח עדכון לשרת על כל דגימת רדיו בודדת (מה שיחנוק את רוחב הפס), כל סוכן מתאמן באופן מקומי למשך מספר אפידודות (שנקראות כאן E), ורק בסוף מעלה את המשקולות הסופיות לשרת.

9.1 למה ממוצע "משוקלל" (Weighted Averaging)?

נניח שמנהל רשת א' אסף 10,000 דגימות ב-Buffer Replay שלו, ומנהל רשת ב' אסף רק 100. כשהשרת עושה ממוצע, הוא לא יכול לתת להם משקל שווה של 50/50. הוא נותן משקל גדול יותר לרשת שלמדה מיותר נתונים. הנוסחה המתמטית של השרת נראית כך:

$$w_{t+1} = \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} w_k^{t+1} \quad (5)$$

כאשר n_k הוא כמות הדגימות של סוכן k .

10 פירוק האלגוריתם צעד אחר צעד

בוא נפרק את האלגוריתם צעד אחר צעד, בצורה הכי ברורה ופרקטית שיש. נראה בדיוק מה קורה **בצד השרת**, מה קורה **בצד הסוכנים** (מנהלי הרשתות), ואיך הלופים האלו עובדים **במקביל**. האלגוריתם מורכב משני חלקים: **אתחול ולופ של סבבי תקשורת**.

10.1 מבט על: מי עושה מה?

זרימת סבב FL אחד --- מי עושה מה?
1. שרת מרכזי (TrainBlock.py): בחירת $S_t \rightarrow$ שידור $w^t \rightarrow$ המתנה $\rightarrow$ אגרגציה w^{t+1}
2. שידור למטה --- כל הסוכנים מקבלים w^t (משקולות, KB)
3. אימון מקומי במקביל (כל הסוכנים בו-זמנית, בלי לדבר ביניהם):
• סוכן $k=1$: Coordinator.py(epochs E על D_1)
• סוכן $k=2$: Coordinator.py(epochs E על D_2)
• סוכן $k=K$: Coordinator.py(epochs E על D_K)

4. העלאה למעלה --- כל סוכן שולח $n_k + w_k$ (משקולות בלבד)

5. אגרגציה בשרת: $w^{t+1} = \sum_{k \in S_t} \frac{n_k}{n} w_k$

טבלה 2: חלוקת אחריות בין שרת לסוכנים

שלב	שרת (TrainBlock)	סוכנים (Coordinator)
אתחול	יוצר w^0 אקראי	מחזיקים Buffer Replay מקומי D_k
סבב t	בוחר S_t , משדר w^t , מחכה, ממצע	מורידים w^t , מתאמנים E פעמים, מעלים w_k
סיום	שומר w^T ב-Train_weights_frl	ממשיכים לפעול עם המודל המעודכן

10.2 שלב 0: האתחול (שרת)

פסאודו-קוד --- אתחול

```
Server initializes  $w^0$ 
```

השרת יוצר את רשת ה-DeepMellow הראשונית עם משקולות אקראיות לחלוטין (נסמן אותן כ- w^0). ברגע זה, הרשת לא יודעת כלום על רדיו קוגניטיבי או על SINR.

בקוד יצירת אובייקט DeepMellow חדש ב-TrainBlock.py. המשקולות נשמרות ב-state_dict של PyTorch.

מה לא קורה אף נתון SINR לא נשלח לשרת. כל סוכן כבר אוסף נתונים מקומית לתוך D_k שלו (Replay Buffer), אבל השרת לא רואה אותם.

10.3 שלב 1: הלופ הגלובלי (שרת)

פסאודו-קוד --- לופ גלובלי (שרת)

```
for each round  $t = 0, 1, 2, \dots, T-1$ :
    Select clients  $S_t$  subset of  $\{1, \dots, K\}$ 
    Broadcast  $w^t$  to all  $k$  in  $S_t$ 
    Wait for uploads  $\{w_k\}$  from all  $k$  in  $S_t$ 
     $w^{t+1} \leftarrow \text{weighted\_average}(\{w_k\}, \{n_k\})$ 
```

השרת מתחיל לנהל את סבבי התקשורת (T). בכל סבב t , הוא בוחר קבוצה של קליינטים (S_t) מתוך כלל הרשתות (K). בפרויקט CARLTON מכיוון שיש מספר קטן של רשתות (למשל $K=3$ רשתות חופפות: סלקום, פרטנר, צבא), השרת יבחר תמיד את כל הרשתות בכל סבב:

$$S_t = \{1, 2, \dots, K\} \quad \text{לכל } t$$

חשוב: בשלב זה השרת רק משדר משקולות וממתין. הוא לא מריץ אימון על SINR.

10.4 שלב 2: הלופ המקומי במקביל (הסוכנים)

פסאודו-קוד --- לופ מקומי (כל סוכן k , במקביל)

```
for each client k in S_t in parallel:
    w_k <- w^t # download global weights
    for epoch e = 1 ... E:
        for batch B from D_k:
            w_k <- w_k - eta * grad(l(w_k; B)) # local SGD step
        upload w_k to server
```

כאן קורה הקסם: הפעולות הבאות קורות במקביל אצל כל מנהלי הרשתות בשטח (סלקום, פרטנר וכו'), מבלי שהם יודעים אחד על השני.

10.4.1 א. הורדת המודל הגלובלי

הורדה

```
w_k <- w^t
```

כל סוכן k מוריד מהשרת את המשקולות הגלובליות הנוכחיות (w^t) וטוען אותן לתוך הרשת שלו. זוהי פקודת `load_state_dict()` ב-PyTorch.

- לפני השלב: לכל הסוכנים אותו "מוח" בדיוק (w^t).
- הנתונים המקומיים (D_k) נשארים על המכשיר --- לא עוברים בערוץ.

10.4.2 ב. לופ האפיזודות / האימון המקומי

אימון מקומי

```
for epoch e = 1 ... E:
    for batch B from D_k:
        w_k <- w_k - eta * grad(l(w_k; B))
```

הסוכן מתנתק מהרשת החיצונית ומתחיל להתאמן "בבית" למשך E פעמים (Epochs).

א. מריץ את רשת הרדיו באמצעות `Coordinator.py`.

ב. שולף באצ'ים (B) של נתוני SINR ותגמולים מתוך ה-Buffer Replay המקומי (D_k).

ג. מחשב את הגרדיאנט של פונקציית ההפסד:

$$\nabla \ell(w_k; B)$$

ד. מעדכן את המשקולות המקומיות:

$$w_k \leftarrow w_k - \eta \nabla \ell(w_k; B)$$

ב-PyTorch זה הצעד הקלאסי של האופטימיזצור: `optimizer.step()`.

בסוף השלב הזה, לכל סוכן יש משקולות קצת שונות (w_k) שמותאמות בדיוק למה שהוא חווה באזור שלו.

למה "במקביל" חשוב?

סוכן 1 יכול להיות ליד שדה תעופה (הפרעות ממכ"ם), סוכן 2 במרכז עיר (עומס סלולרי), וסוכן 3 בצבא (דפוסי שידור שונים). כולם מתאמנים בו-זמנית על D_k שונה --- אבל כולם מתחילים מאותו w^t .

10.4.3 ג. העלאה לשרת

כל סוכן מסיים את ה- E אפיזודות שלו ושולח רק את המשקולות המעודכנות (w_k) חזרה לשרת המרכזי. גודל ההעלאה: עשרות--מאות KB (משקולות, DeepMellow לא GB של מדידות SINR).

10.5 שלב 3: האגרציה / שילוב התובנות (שרת)

פסאודו-קוד --- אגרציה (שרת)

```
w^{t+1} <- sum over k in S_t of (n_k / sum_j n_j) * w_k
```

השרת חיכה שכולם יסיימו ויעלו את המשקולות שלהם. כעת הוא מבצע את הממוצע המשוקלל (Weighted Average) כדי לייצר את המודל של הסבב הבא (w^{t+1}):

$$w^{t+1} = \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} w_k \quad (6)$$

- רשת עם יותר דגימות (n_k גדול) משפיעה יותר על המודל הגלובלי.

- אחרי האגרציה, w^{t+1} הופך ל- w^t של הסבב הבא.

- הלופ חוזר עד $t = T - 1$.

10.6 האלגוריתם המלא במבט אחד

FedAvg --- אלגוריתם מלא

```
# --- Initialization (Server) ---
Server initializes w^0

# --- Communication Rounds ---
for each round t = 0, 1, 2, ..., T-1:

    # Server: client selection
    Select clients S_t subset of {1, ..., K}

    # Clients: parallel local training
    for each client k in S_t in parallel:
        w_k <- w^t
        for epoch e = 1 ... E:
            for batch B from D_k:
                w_k <- w_k - eta * grad(l(w_k; B))
        upload w_k to server

    # Server: aggregation
    w^{t+1} <- sum over k in S_t of (n_k / sum_j n_j) * w_k

# --- End ---
Server saves w^T
```


שלב 2א --- הורדת מודל (ס)
מי: סוכן k | **פעולה:** $\text{load_state_dict}()$
תקשורת: ---

שלב 2ב --- אימון מקומי (י)
מי: סוכן k | **פעולה:** $\text{load_state_dict}()$
פירוט: לולאה על באצייס k
תקשורת: --- (הסוכנים)

שלב 2ג --- העלאה (סוכן k)
מי: סוכן $k \rightarrow$ שרת
פירוט: שליחת המשקולות
תקשורת: משקולות $w_k +$

שלב 3 --- אגרגציה (שרת)
מי: שרת | **פעולה:** $\text{load_state_dict}()$
פירוט: ממתין שכל $k \in S_t$
תקשורת: כלום

הלופ במקביל --- מה זה אומר?
בשלב 2, כל מנהלי השרתים
שלהם. הם לא מתקשרים ביניהם
ומחכה" עד שכולם מסיימים

דוגמה מספרית לסבב אחד
1. שרת שולח w^{42} לשל
2. סלקום מתאמן 5 אפי
3. פרטנר מתאמן 5 אפי
4. צבא מתאמן 5 אפיו
5. שרת מחשב: $w_3 = \frac{000}{000}$

11 הדילמה הגדולה:

כאן אתה כמפתח צריך לקבל
• E גדול (הרבה אימון מ
שגויות" שמתאימות ר
• E קטן (למשל, סבב אח
של משקולות ברשת הפ

12.1 מוטיבציה: למה FedAvg

כפי שראינו, ב- FedAvg , אם
ימשוך את המשקולות שלהן)
התוצאה תהיה מודל גרוע שלא
 FedProx פותר את זה על

FedAvg כל סוכן נ

FedProx מוסיף אי
רחוק מדי

דוגמה מ-CARLTON:

בערוץ 4) יפתחו גרדיאנטים ש
מודל ``אמצעי״ גרוע. ב- dProx

12.2 פונקציית המטרה ה

במקום שסוכן k ינסה רק ל
המשוואה הבאה:

7)

בוא נבין את שני החלקים

$F_k(w)$ פונקצי
על נתו

$\frac{\mu}{2} \|w - w^t\|^2$ האיבר
בין המ
הסבב

12.2.1 מה עושה הפרמטר μ

μ הוא כפתור ה״ווליום״ של ו

μ ערך	
הא	$\mu = 0$
ה	μ ענק
μ מכוון נכון	

אינטואיציה מתמטית: ה

משיכה חזרה לעוגן הגלובלי:

8)

חשוב על w^t כעוגן (anchor).
עדכונים סותרים כשהגרדיאנט

FedProx --- אלגוריתם מלא (ההבדל מסומן ב-*)

```

n (Server) ---
w^0

Rounds ---
0, 1, 2, ..., T-1:

    t selection + broadcast
    S_t subset of {1, ..., K}
    for all k in S_t

        parallel local training
        for k in S_t in parallel:

            e = 1 ... E:
                for each B from D_k:
                    FedProx only: proximal term pulls w_k toward w^t
                    w_k <- w_k - eta * (grad(F_k(w_k; B)) + mu * (w_k - w^t))
                send w_k to server

Aggregation (identical to FedAvg)
w^{t+1} = sum over k in S_t of (n_k / sum_j n_j) * w_k

```

מה משתנה בקוד CARLTON?

רק שורת העדכון ב-`DeepMellow.learn()` / האופטימיזצור: במקום `loss.backward()` את האיבר $\frac{\mu}{2} \|w - w^t\|^2$ ל-`loss` לפני ה-`backward`. האגרציה ב-`TrainBlock.py` נשארת ללא שינוי.

12.6 השתתפות חלקית --- פירוט נוסף

בעיה ב-FedAvg כשמשתתף רק חלק מהרשתות, עדכונים ממקורות שונים בכל סבב יציב.

יתרון FedProx העונש הקרבה מייצב את האימון גם כשהשתתפות מדורבנת --- ואלחוטיות.

$F_k(w) + \frac{\eta}{2} \ w - w^*\ ^2$	$\min_k F_k(w)$	אימון מקומי
$\mu \geq 0$	---	פרמטר נוסף
עמיד יותר	רגיש	Drift / Non-IID
יציב יותר	פחות יציב	השתתפות חלקית

13 החיבור לקוד שלך

בסעיף ההטמעה הספציפית, מוזכרת ההטמעה ב-BuildingBlocks/TrainBlock.py. הפונקציה שתיצור ב-PyTorch תיקח את ה-state_dict (מילון המשקולות של PyTorch) מכל מנהלי הממוצע המשוקלל הזה, ולאחר 1000 סבבים תשמור את התוצאה בתיקייה weights_frl/ מכיוון שאצלך כל אפיזודה יכולה להסתיים בזמן קצת שונה או עם כמות צעדים שונה סוכן עשויה להשתנות.